

Reto 2 — Patrones de Diseño Arquitectónico

Segunda evolución del sistema: del diseño correcto al diseño robusto

Sistema: Hacienda
Curso: Arquitectura de Software
Docente: César Augusto López

- Integrantes:**
- María Alejandra Vargas Duque
 - Mateo Rojas Hernández
 - David Salcedo Higueta

1. Análisis de puntos de dolor de la arquitectura actual

El Reto 1 dejó una arquitectura que cumplía los principios SOLID con algunas fallas. En este reto analizamos qué tan sencillo sería evolucionar esa arquitectura cuando aparezcan nuevas necesidades del negocio.

El análisis se concentra en lugares donde el sistema funciona correctamente, pero un cambio obliga a modificar demasiados elementos, repetir lógica o conocer detalles que deberían estar concentrados en un solo lugar.

El costo se mide contando las clases y archivos que actualmente sería necesario modificar para realizar cada cambio. La prioridad se determina considerando el impacto del cambio, la frecuencia esperada y el riesgo de introducir regresiones.

1.1. Puntos de dolor identificados

ID	Dónde	Qué lo hace rígido o caro	Costo hoy	Prioridad	Origen
P-01	FabricaRes , CatalogoRes , ParametrosRes , GestorReses , TipoRes	Agregar un subtipo exige editar el diccionario de creadores, configuración, switch , parámetros, contadores y enum, además de crear la nueva clase.	6 archivos / 2 capas por subtipo	Alta	Propio
P-02	IVacunaFactory , FabricaVacuna , VacunaController	La decisión de categoría está repartida entre métodos por tipo, un ternario por cadena en el controlador y cadenas repetidas en la vista.	≈8 clases / 5 archivos por categoría	Alta	Propio
P-03	Venta , FabricaVenta , RepositorioVentaSqlite	Venta nace acoplada a una sola res. SC-1 exige pasar a una venta que pueda contener varios elementos.	≈6 clases / 6 archivos	Alta	Propio
P-04	Validador* , fábricas y servicios	El proceso de fabricar y luego validar está repetido. Además, existen umbrales diferentes para reglas que deberían ser iguales.	4 validadores / 4 archivos	Media	Propio
P-05	RepositorioVentaSqlite	Al leer una venta se vuelve a crear la res con un identificador nuevo, por lo que la identidad de la entidad no se conserva.	1 archivo que afecta las lecturas	Media	Propio
P-06	DomainEventPublisherConsola , DomainEvents	Las reacciones están ligadas al destino por consola. Agregar un nuevo consumidor obliga a intervenir el mecanismo actual de publicación.	1 clase que bloquea N consumidores	Media	Asistido por IA
P-07	AutorizadorRbca y políticas	Está registrado y funciona, pero actualmente no participa en el flujo real.	—	No se interviene	Asistido por IA

ID	Dónde	Qué lo hace rígido o caro	Costo hoy	Prioridad	Origen
P-08	Program.cs	El punto de ensamblaje tiene efectos de arranque y su orden puede modificar el comportamiento.	—	No se interviene	Asistido por IA
P-09	Res y Vacuna (subtipos)	Serializar() está repetido en cinco subtipos con el mismo formato de tubería.	5 implementaciones / 5 archivos	No se interviene	Asistido por IA
P-10	ServicioVacunacion	Existen dos procesos casi idénticos y varios métodos que repiten la misma estructura.	4 métodos / 1 archivo	Media	Asistido por IA
P-11	TipoRes ↔ TipoPotrero	Existen enums paralelos y un switch de mapeo. Un subtipo nuevo requiere modificar ambos y el mapeo.	3 puntos adicionales	Baja	Propio y asistido por IA
P-12	Las cuatro Fabrica*	Las fábricas resuelven problemas similares de maneras diferentes: diccionario, método por tipo, reloj en la firma o solamente constructor.	4 clases / 4 archivos	Alta	Asistido por IA
P-13	GestorReses.AgregarRes vs. AlimentarRes	Algunos pares de evento + mensaje están duplicados entre flujos. Una nueva reacción debe escribirse en más de un lugar.	2 flujos / 1 archivo	Media	Asistido por IA
P-14	Dinero	La moneda COP está definida directamente en el código.	1 literal	No se interviene	Asistido por IA

1.2. Puntos de dolor que no se intervienen

En este ciclo no todos los puntos de dolor serán modificados. El criterio utilizado es que el costo y el riesgo de realizar el cambio deben estar justificados por el beneficio que aporta a las solicitudes de cambio actuales.

ID	Argumento: el remedio cuesta más que el problema
P-05	La identidad inestable durante la rehidratación debe corregirse, pero su solución no es necesaria para implementar SC-1. Se registra como deuda para no ampliar innecesariamente el alcance.
P-07	Activar RBAC cambiaría el comportamiento observable relacionado con autorización y penalizaciones. SC-1 no necesita modificar esta parte.
P-08	Reorganizar el arranque puede alterar el seeder y otros efectos secundarios. En este ciclo se conservan los registros existentes y únicamente se agregan los nuevos al final.
P-09	Eliminar la duplicación de Serializar() sería una mejora válida, pero no resuelve directamente SC-1 y podría afectar el formato de persistencia existente.
P-14	La hacienda trabaja actualmente con una única moneda. Convertirla en una configuración general no responde a ninguna solicitud de cambio del Anexo B.

Los puntos de dolor que sí serán intervenidos son principalmente **P-01, P-03, P-04, P-06, P-10, P-12 y P-13**, debido a que presentan una relación directa con las solicitudes de cambio seleccionadas y tienen un costo de modificación que justifica aplicar una solución arquitectónica.

2. Decisión de patrones a incorporar

Se evaluaron los patrones disponibles y se seleccionaron cuatro para formar parte del diseño TO-BE:

- Factory Method
- Template Method
- Builder
- Observer

La selección no se hizo únicamente porque los patrones sean conocidos o considerados buenas prácticas. Cada patrón está asociado con uno o más puntos de dolor identificados anteriormente.

2.1. Factory Method

Campo	Contenido
Patrón evaluado	Factory Method
Familia	Creacional
Punto de dolor que atacaría	P-01, P-02 y P-12 , principalmente. También contribuye a reducir parte del acoplamiento identificado en P-11.
Qué gana y qué cuesta	Gana: agregar un nuevo tipo requiere concentrar la creación en un creador específico y un registro. Cuesta: una clase adicional por tipo y una indirección adicional entre quien solicita la creación y el objeto creado.
Decisión	Adoptado
Por qué	P-01 tiene un costo medido de 6 archivos y 2 capas por subtipo. Factory Method permite reducir ese cambio a un nuevo creador y su registro, manteniendo fuera de los servicios la decisión concreta de creación.

2.2. Template Method

Campo	Contenido
Patrón evaluado	Template Method
Familia	De comportamiento
Punto de dolor que atacaría	P-04 y P-10 , donde se repite la misma secuencia de fabricación y validación.
Qué gana y qué cuesta	Gana: una única secuencia común para procesos que actualmente están duplicados. Cuesta: introduce una estructura de herencia y un marco común que puede aumentar la complejidad de depuración.
Decisión	Adoptado
Por qué	P-10 presenta procesos gemelos de 27 líneas que solo difieren en tres. Es un indicador concreto de que existe una secuencia común que puede centralizarse.

2.3. Builder

Campo	Contenido
Patrón evaluado	Builder
Familia	Creacional
Punto de dolor que atacaría	P-03 , especialmente el acoplamiento de <code>Venta</code> a una única res.
Qué gana y qué cuesta	Gana: permite construir una venta con varios elementos sin convertir el constructor en una lista creciente de parámetros. Cuesta: agrega un objeto Builder y un estado intermedio durante la construcción.
Decisión	Adoptado
Por qué	SC-1 requiere vender reses junto con productos derivados. El diseño actual de <code>Venta</code> tendría que modificarse en aproximadamente seis clases/archivos. Builder concentra la construcción de la venta multi-ítem.

2.4. Observer

Campo	Contenido
Patrón evaluado	Observer
Familia	De comportamiento
Punto de dolor que atacaría	P-06 y P-13 , relacionados con las reacciones ante eventos.
Qué gana y qué cuesta	Gana: permite agregar nuevos consumidores de eventos sin modificar el proceso que produjo el evento. Cuesta: aumenta el número de participantes y requiere definir un orden determinista cuando varias reacciones deben

Campo	Contenido
	ejecutarse.
Decisión	Adoptado
Por qué	Actualmente una reacción nueva obliga a intervenir el publicador o repetir código en los flujos. Observer permite separar el hecho ocurrido de las acciones que deben reaccionar a él.

2.5. Patrones descartados

Patrón	Familia	Punto relacionado	Decisión	Por qué
Composite	Estructural	P-03	Descartado	Una venta de SC-1 es una lista plana de elementos. Composite tendría sentido si existieran grupos o productos compuestos dentro de otros grupos.
Abstract Factory	Creacional	P-01	Descartado	No existen familias de objetos que deban crearse conjuntamente. Agregar este patrón introduciría una abstracción que no responde a una necesidad actual.
Prototype	Creacional	—	Descartado	Los objetos se crean con datos nuevos. Copiarlos podría compartir estado mutable y no resolvería ninguno de los puntos prioritarios.
Singleton	Creacional	—	Descartado	La unicidad de los componentes ya se controla desde <code>Program.cs</code> . Un acceso global aumentaría el acoplamiento y dificultaría las pruebas.
Adapter	Estructural	—	Descartado	El acceso a SQLite ya está aislado mediante interfaces de repositorio.
Bridge	Estructural	—	Descartado	La separación entre abstracción e implementación ya está cubierta por las interfaces existentes.
Decorator	Estructural	—	Descartado	No existe una responsabilidad que necesitemos añadir dinámicamente a los objetos.
Facade	Estructural	—	Descartado	Los servicios actuales ya ofrecen puntos de acceso a las operaciones. Otra fachada podría terminar acumulando lógica de negocio y romper SRP.
Flyweight	Estructural	—	Descartado	El tamaño esperado del dominio no genera un problema de memoria que justifique compartir objetos.
Proxy	Estructural	P-07	Descartado	RBAC está congelado y no existe una necesidad de carga diferida.
Chain of Responsibility	Comportamiento	P-04	Descartado	Los pasos del proceso tienen un orden fijo. Template Method representa mejor la secuencia completa.
Command	Comportamiento	—	Descartado	No existe una necesidad de deshacer acciones ni de poner comandos en cola.
Interpreter	Comportamiento	—	Descartado	Las reglas actuales son demasiado simples para justificar una gramática propia.
Iterator	Comportamiento	—	Descartado	Los mecanismos de recorrido del lenguaje ya cubren esta necesidad.
Mediator	Comportamiento	P-06	Descartado	Centralizaría la comunicación en un objeto adicional, mientras Observer permite mantener separadas las reacciones.
Memento	Comportamiento	—	Descartado	No existe una solicitud de recuperación de estados anteriores.
State	Comportamiento	—	Descartado	Las transiciones del chip ya están representadas de manera declarativa.
Strategy	Comportamiento	P-02 y P-10	Descartado	El problema principal es seleccionar y crear el tipo, no intercambiar algoritmos durante la ejecución.

2.6. Bitácora de IA

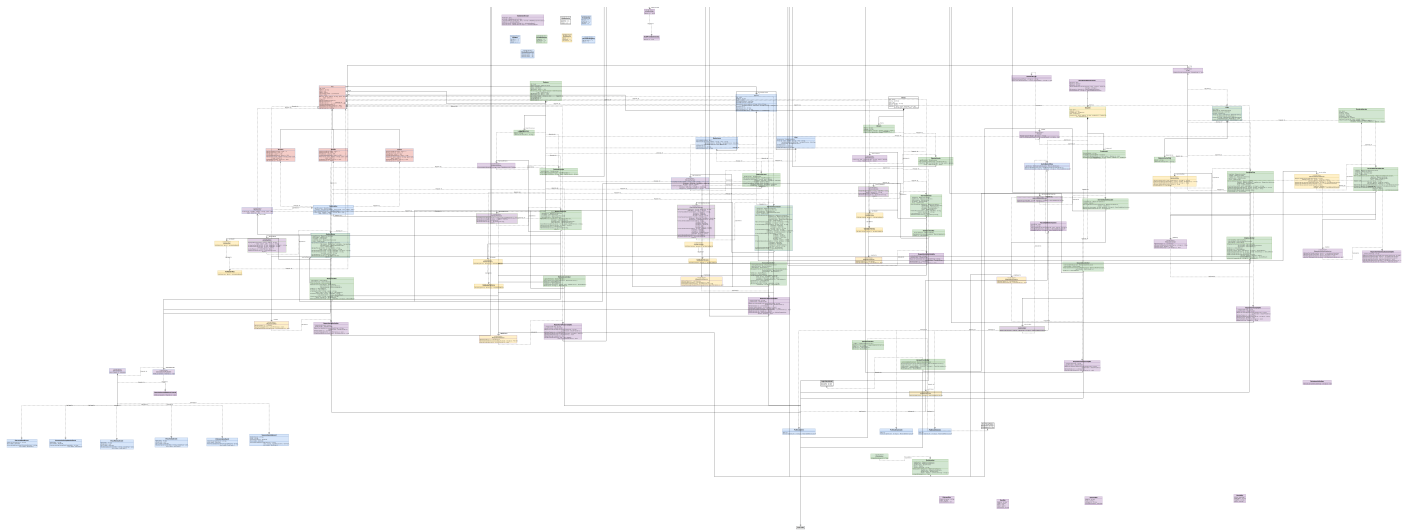
ID	Qué consultaron	Qué propuso la herramienta	Qué hicieron	Argumento propio y evidencia
B-01	Análisis del diseño existente.	Identificar lugares donde agregar un tipo nuevo requiere modificar varias capas.	Aceptamos.	Se verificó que P-01 requiere aproximadamente 6 archivos y 2 capas.
B-02	Organización del trabajo.	Relacionar cada sección con un criterio de la rúbrica.	Aceptamos.	Permite verificar que cada actividad tenga su evidencia correspondiente.
B-03	Alcance del cambio.	Incluir las vistas como objetivo principal de modificación.	Rechazamos.	Las vistas deben conservarse; se consideran costo cuando una solicitud las afecta, pero no son el objetivo principal del reto.
B-04	Clasificación de hallazgos.	Tratar todos los problemas encontrados como cambios obligatorios.	Corregimos.	El equipo decidió separar los problemas que realmente justifican intervención de la deuda técnica.
B-05	Selección entre las solicitudes de cambio.	Recomendar SC-1 por tener mayor contraste entre AS-IS y TO-BE.	Aceptamos.	SC-1 afecta P-01 y P-03, dos de los puntos con mayor costo medido.
B-06	Detección de puntos de dolor.	Medir el costo mediante clases y archivos.	Aceptamos.	Coincide con el criterio explícito de la actividad 1.
B-07	Evaluación de patrones.	Evaluar más patrones que el mínimo.	Aceptamos.	Permite justificar mejor los descartes y demostrar criterio de selección.
B-08	Selección de patrones.	Factory Method, Template Method, Builder y Observer.	Aceptamos.	Los cuatro tienen puntos de dolor concretos y el total se mantiene dentro del límite de patrones adoptados.
B-09	Diseño TO-BE.	Utilizar registros para conectar tipos con creadores.	Aceptamos.	El mecanismo permite centralizar la asociación sin volver a introducir un switch por tipo.
B-10	Implementación de la nueva vaca lechera.	Crear <code>FabricaVacaLechera</code> .	Aceptamos.	Permite demostrar de forma medible el objetivo de reducir el costo de agregar un subtipo.
B-11	Diagramación.	Separar visualmente lo que sale de lo que entra.	Aceptamos.	Facilita verificar la trazabilidad entre AS-IS y TO-BE.
B-12	Validación del diseño.	Revisar el comportamiento antes y después.	Aceptamos.	Es necesario para garantizar que los patrones no cambien el comportamiento observable.
B-13	Diagramas de evolución.	Mantener una disposición comparable entre AS-IS y TO-BE.	Aceptamos.	Permite identificar visualmente qué elementos se transforman y cuáles permanecen.
B-14	Deuda del Reto 1.	Eliminar responsabilidades que permanecían mezcladas.	Aceptamos con ajustes.	Solo se incorporan los cambios compatibles con el alcance del Reto 2.
B-15	Repetición de reglas.	Buscar una única fuente para reglas comunes.	Aceptamos.	Reduce el riesgo de que dos lugares terminen utilizando diferentes valores para la misma regla.
B-16	Revisión de la validación de edad.	Cuestionar la regla existente.	Fue idea nuestra.	La revisión del historial permitió confirmar una regresión y recuperar el comportamiento esperado.

3. Diseño del TO-BE

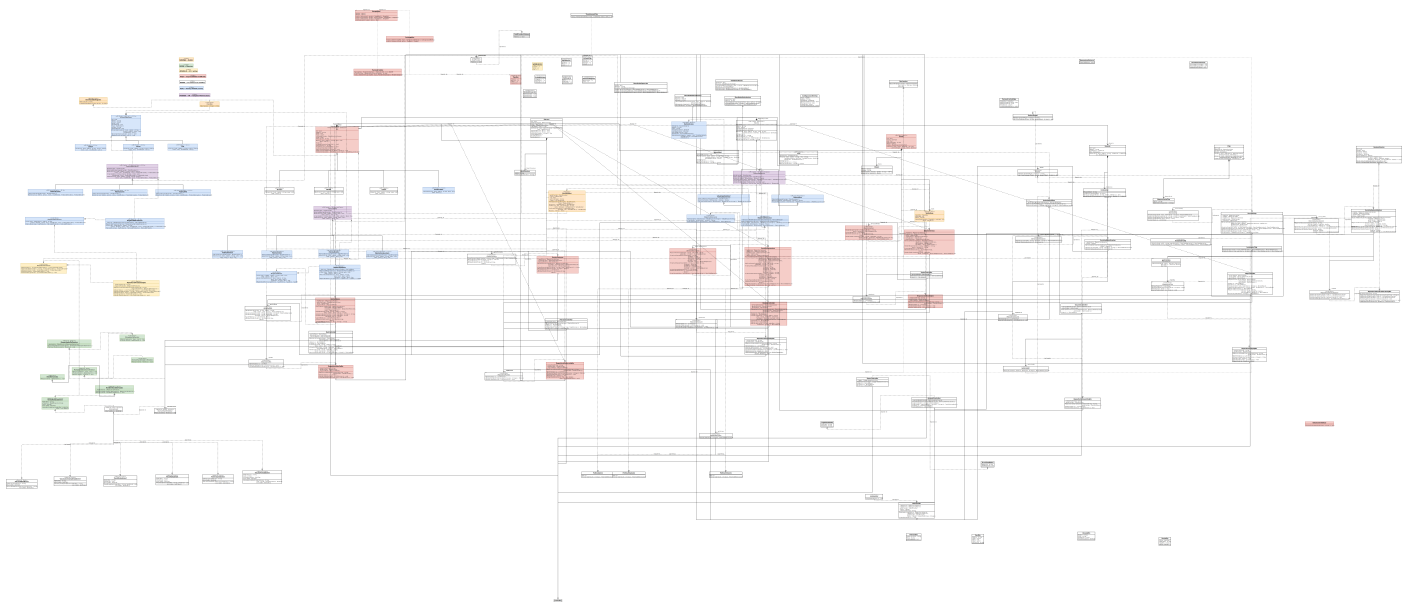
El diseño TO-BE mantiene el estilo arquitectónico existente. No se migran capas, no se introducen frameworks de inyección automática y no se cambia el comportamiento observable.

El cambio se concentra en cómo se crean los objetos, cómo se construye una venta y cómo se distribuyen las reacciones ante eventos.

3.1. Diagrama AS-IS



3.2. Diagrama TO-BE



- FabricaDeRes y creadores concretos → **Factory Method**;
- RegistroDeReses → registro de creadores;
- FabricaDeVacuna y creadores → **Factory Method + Template Method**;
- VentaBuilder → **Builder**;
- IVendible → contrato común de elementos vendibles;
- DespachadorDeEventos → **Observer**;
- HandlerConsola y nuevos handlers → observadores;
- FabricaVacaLechera → nuevo creador concreto.

3.3. Tabla de cambio estructural

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía de él y cómo se reconecta
E-01	FabricaRes + IResFactory	Se transforma	Concentraba decisiones de creación y utilizaba estructuras que debían modificarse al agregar tipos.	Define la estructura común para los creadores concretos.	Los servicios pasan a utilizar RegistroDeReses , que selecciona el creador correspondiente.

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía de él y cómo se reconecta
E-02	CatalogoRes	Se transforma	Contenía configuración y decisiones sobre creación y mapeo.	Conserva el procesamiento necesario, mientras la creación se delega al registro.	Repositorios y servicios se conectan mediante RegistroDeReses .
E-03	IVacunaFactory / FabricaVacuna	Se transforma	La creación estaba dividida en métodos por tipo.	Utiliza una estructura común y creadores concretos.	ServicioVacunacion y VacunaController proporcionan los datos necesarios.
E-04	Validadores	Se transforma	Validaban los objetos después de haberlos construido.	Las reglas comunes forman parte del proceso común de construcción.	Los servicios dejan de coordinar validadores independientes.
E-05	GestorReses	Se transforma	Concentraba estadísticas y reacciones.	Mantiene las consultas estadísticas, mientras las reacciones pasan a handlers.	Los controladores siguen consumiendo las mismas claves de estadísticas.
E-06	RepositorioVentaSqlite	Se transforma	Rehidrataba una res con un identificador nuevo.	La rehidratación conserva el identificador persistido.	Las lecturas siguen llegando desde los servicios y pueden correlacionarse con la venta original.
E-07	DomainEventPublisherConsole	Se transforma	Enviaba directamente los eventos a consola.	Se convierte en un despachador que notifica a distintos handlers.	Los productores continúan publicando eventos; los handlers reciben las notificaciones.
E-08	Venta + FabricaVenta	Se transforma	Una venta estaba asociada a una sola res.	Una venta puede contener varios elementos IVendible .	ServicioVentas utiliza VentaBuilder para construir la venta.
E-09	Lotes en ServicioVacunacion	Se transforma	Había dos procesos casi idénticos.	El proceso común se centraliza mediante Template Method.	El servicio proporciona los datos específicos y la fábrica ejecuta la secuencia común.
E-10	Serializar() en subtipos	Se mantiene	Cada subtipo contiene una implementación repetida.	Se mantiene sin intervenir en este ciclo.	No se modifica ninguna dependencia.
E-11	RegistroDeReses , RegistroDeVacunas , RegistroDeProductos	Entra	No existía un punto único para relacionar tipos con creadores.	Centralizan estas asociaciones.	Servicios y repositorios consultan los registros.
E-12	Productos derivados + IVendible	Entra	Los productos derivados no formaban parte de una venta multi-ítem.	Lácteos, carne y piel pueden participar en una venta.	VentaBuilder trabaja contra IVendible , no contra cada producto concreto.
E-13	FabricaVacaLechera	Entra	No existía el nuevo subtipo.	Crea la nueva variedad de res.	Se registra en RegistroDeReses .
E-14	Program.cs	Se transforma	Ensamblaba los componentes existentes y el orden de registro era sensible.	Conserva los registros existentes y agrega los nuevos al final.	Sigue siendo el único punto donde se conoce el ensamblaje completo.

3.4. Ficha por patrón adoptado

3.4.1. Factory Method

Campo	Contenido
Patrón y punto de dolor que resuelve	Factory Method. Resuelve principalmente P-01 , además de P-02 y P-12. Evidencia: <code>FabricaRes</code> , <code>CatalogoRes</code> , <code>GestorReses</code> y <code>TipoRes</code> ; el costo medido de P-01 es 6 archivos y 2 capas por subtipo.
Alternativas que evaluaron	1. No hacer nada: mantener las modificaciones distribuidas. 2. Simple Factory: centralizaría la decisión, pero conservaría un condicional creciente. 3. Abstract Factory: no se justifica porque no existen familias de productos que deban crearse conjuntamente.
Qué sale y qué entra	Sale: el diccionario central y las decisiones distribuidas de creación. Entra: <code>FabricaDeRes</code> , creadores concretos y <code>RegistroDeReses</code> .
Cómo se relaciona	<code>RegistroDeReses</code> selecciona el creador. El creador concreto construye la res. <code>Template Method</code> proporciona la secuencia común cuando corresponde. Los servicios consumen el resultado sin conocer la clase concreta que lo construyó.
Impacto	Se crea un creador por subtipo y se modifica el registro. Se modifican las fábricas existentes y <code>Program.cs</code> para registrar los componentes. En SC-1 se agrega <code>FabricaVacaLechera</code> . El cambio esperado para un nuevo subtipo pasa de aproximadamente 6 archivos a 1 clase + 1 registro .
Qué cuesta	Se agregan clases y una capa de indirección. Para localizar el código de creación, un desarrollador debe seguir el registro hasta el creador concreto.
Origen	Propuesta de la herramienta aceptada y ajustada por el equipo. Relacionada con B-08, B-09 y B-10 .

3.4.2. Template Method

Campo	Contenido
Patrón y punto de dolor que resuelve	Template Method. Resuelve principalmente P-04 y P-10 . La evidencia de P-10 son dos lotes gemelos de 27 líneas que difieren solamente en tres.
Alternativas que evaluaron	1. No hacer nada: continuar duplicando el proceso. 2. Chain of Responsibility: separar validadores en una cadena. 3. Template Method: mantener una secuencia común y dejar variables los datos específicos.
Qué sale y qué entra	Sale: duplicación de procesos y validadores independientes que repiten reglas. Entra: una estructura común para validar, construir y completar el proceso.
Cómo se relaciona	Las fábricas concretas proporcionan los datos específicos. <code>Factory Method</code> decide qué creador utilizar. <code>Template Method</code> controla el orden general. <code>Observer</code> puede recibir el evento producido al terminar el proceso.
Impacto	Se modifica <code>ServicioVacunacion</code> y las fábricas relacionadas. Se reducen cuatro validadores y se centralizan reglas comunes. Los casos existentes de creación de vacunas deben producir exactamente los mismos mensajes.
Qué cuesta	Se introduce herencia y una secuencia común. Además, existe una tensión con LSP si una subclase no puede cumplir uno de los pasos. Para compensarlo, los puntos variables se representan mediante datos y no mediante pasos vacíos.
Origen	Propuesta de la herramienta aceptada con ajustes. Relacionada con B-08, B-14 y B-15 .

3.4.3. Builder

Campo	Contenido
Patrón y punto de dolor que resuelve	Builder. Resuelve P-03 , donde <code>Venta</code> está acoplada a una sola res. La evidencia es un costo aproximado de 6 clases/archivos para adaptar la venta a SC-1.
Alternativas que evaluaron	1. No hacer nada: continuar modificando <code>Venta</code> , fábrica, repositorio y vista. 2. Constructor con más parámetros: permitir todos los elementos directamente en <code>Venta</code> . 3. Composite: representar grupos de elementos. 4. Builder: construir la venta progresivamente.
Qué sale y qué entra	Sale: la construcción de una venta centrada exclusivamente en una res. Entra: <code>VentaBuilder</code> e <code>IVendible</code> , junto con productos derivados que implementan ese contrato.
Cómo se relaciona	<code>ServicioVentas</code> utiliza <code>VentaBuilder</code> . El Builder recibe objetos <code>IVendible</code> . Los creadores de productos pertenecen a <code>Factory Method</code> .
Impacto	<code>Venta</code> se modifica para representar varios elementos. Se crea <code>VentaBuilder</code> y se incorporan las clases de productos derivados. SC-1 puede representar una venta con res, lácteos, carne o piel sin modificar el constructor por cada combinación.
Qué cuesta	Aparece un objeto adicional y un estado intermedio de construcción. Los errores durante la construcción pueden ser más difíciles de rastrear si no se valida correctamente al finalizar.

Campo	Contenido
Origen	Propuesta de la herramienta aceptada. Relacionada con B-05, B-08 y B-09 .

3.4.4. Observer

Campo	Contenido
Patrón y punto de dolor que resuelve	Observer. Resuelve P-06 y P-13 , donde las reacciones están ligadas a los flujos que producen los eventos.
Alternativas que evaluaron	1. No hacer nada: continuar agregando reacciones directamente a los flujos. 2. Llamadas directas: cada productor llama a todos los consumidores. 3. Mediator: centralizar la coordinación. 4. Observer: publicar el evento y registrar consumidores independientes.
Qué sale y qué entra	Sale: la dependencia directa entre productor y reacción. Entra: <code>DespachadorDeEventos</code> , <code>HandlerConsole</code> y handlers adicionales como <code>HandlerStockDerivados</code> .
Cómo se relaciona	Los servicios producen eventos. El despachador los distribuye. Los handlers reaccionan. Factory Method y Builder producen objetos o ventas que posteriormente pueden generar eventos.
Impacto	Se modifica el publicador y se crean los handlers. Agregar una reacción nueva deja de requerir modificar el productor. Para SC-1 se puede incorporar una reacción relacionada con stock sin modificar el flujo de venta.
Qué cuesta	Aumenta el número de participantes y la trazabilidad durante la depuración. Se debe definir explícitamente el orden de los handlers para conservar las salidas actuales.
Origen	Propuesta de la herramienta aceptada y ajustada. Relacionada con B-08, B-09 y B-12 .

4. Verificación de SOLID y del comportamiento

4.1. Matriz de verificación

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Factory Method	Refuerza	Refuerza	Neutro	Neutro	Refuerza
Template Method	Refuerza	Refuerza	Tensionado pero compensado	Neutro	Neutro
Builder	Refuerza	Refuerza	Refuerza	Refuerza	Refuerza
Observer	Refuerza	Refuerza	Neutro	Refuerza	Refuerza

4.2. Factory Method

- **SRP:** la creación queda separada de los servicios.
- **OCP:** agregar un tipo nuevo no requiere ampliar un `switch` central.
- **DIP:** los consumidores dependen de abstracciones y registros, no de creadores concretos.

4.3. Template Method

- **SRP:** centraliza una secuencia que estaba repetida.
- **OCP:** permite extender el proceso mediante nuevos creadores.
- **LSP — tensión:** una subclase no puede implementar correctamente un paso obligatorio si el paso no corresponde a su naturaleza.
- **Compensación:** los elementos variables se representan mediante datos y no mediante métodos vacíos.

4.4. Builder

- **SRP:** separa la construcción de la representación de una venta.
- **OCP:** agregar un nuevo elemento vendible no obliga a crear otra variante del constructor.
- **ISP:** `IVendible` contiene únicamente el comportamiento necesario para participar en una venta.
- **DIP:** `VentaBuilder` trabaja con `IVendible`.

4.5. Observer

- **SRP**: cada handler tiene una responsabilidad de reacción.
- **OCP**: se pueden agregar handlers sin modificar el productor.
- **ISP**: los consumidores reciben únicamente la interfaz/evento que necesitan.
- **DIP**: el productor depende del mecanismo abstracto de publicación.

4.6. Demostración de SOLID

Caso 1 — Factory Method

Campo	Contenido
Patrón adoptado	Factory Method
SRP	La responsabilidad de crear cada subtipo queda separada de los servicios que utilizan las reses.
OCP	Agregar una nueva variedad implica crear un nuevo creador y registrarlo, evitando ampliar un <code>switch</code> central.
LSP	Todos los creadores producen objetos que cumplen el contrato esperado de <code>Res</code> .
ISP	Los consumidores utilizan contratos específicos y no necesitan conocer todos los detalles de creación.
DIP	Los servicios dependen de abstracciones y registros, no de una clase concreta para cada subtipo.

Caso 2 — Observer

Campo	Contenido
Patrón adoptado	Observer
SRP	Cada handler tiene una responsabilidad concreta frente al evento que recibe.
OCP	Una nueva reacción puede incorporarse mediante otro handler sin modificar el productor del evento.
LSP	Los handlers cumplen el contrato definido para recibir el evento.
ISP	Cada consumidor recibe únicamente el contrato necesario para procesar la notificación.
DIP	Los productores dependen del mecanismo abstracto de publicación y no de cada consumidor concreto.

4.8. Evidencia de comportamiento

Se deben ejecutar los casos del Reto 1 y los nuevos casos relacionados con los patrones. Las salidas antes y después deben poder compararse y coincidir, salvo los cambios explícitamente autorizados por la solicitud SC-1.

ID	Caso	Qué demuestra
C-01	Agregar una res válida	Conservación del comportamiento anterior.
C-02	Edad no válida	Conservación de validaciones y mensajes.
C-03	Res duplicada	Conservación de reglas existentes.
C-04	Vacuna con esquema incompleto	Conservación del proceso de vacunación.
C-05	Lote ya aplicado	Conservación de la regla existente.
C-06	Máximo por categoría	Conservación de límites.
C-07	Crear lote de vacunas	Template Method y conservación de numeración.
C-08	Venta de una res	Compatibilidad con el comportamiento anterior.
C-09	Búsqueda inexistente	Conservación de mensajes.
C-10	Instalar chip	Conservación del comportamiento del Reto 1.
C-11	Cambio de estado de chip	Conservación de transiciones.
C-12	Alimentar res	Observer y conservación de eventos.
C-13	Leer la misma venta dos veces	Comprobación de identidad durante la lectura.

ID	Caso	Qué demuestra
C-14	Venta de res + derivados	Demostración de SC-1 y Builder.
C-15	Venta con monto 0	Comprobación de regla única de validación.
C-16	Venta que genera consola + stock	Demostración de Observer y orden de handlers.
C-17	Vaca lechera en el proceso completo	Demostración de Factory Method + Template Method.
C-18	Persistir y leer diferentes subtipos	Compatibilidad con persistencia.

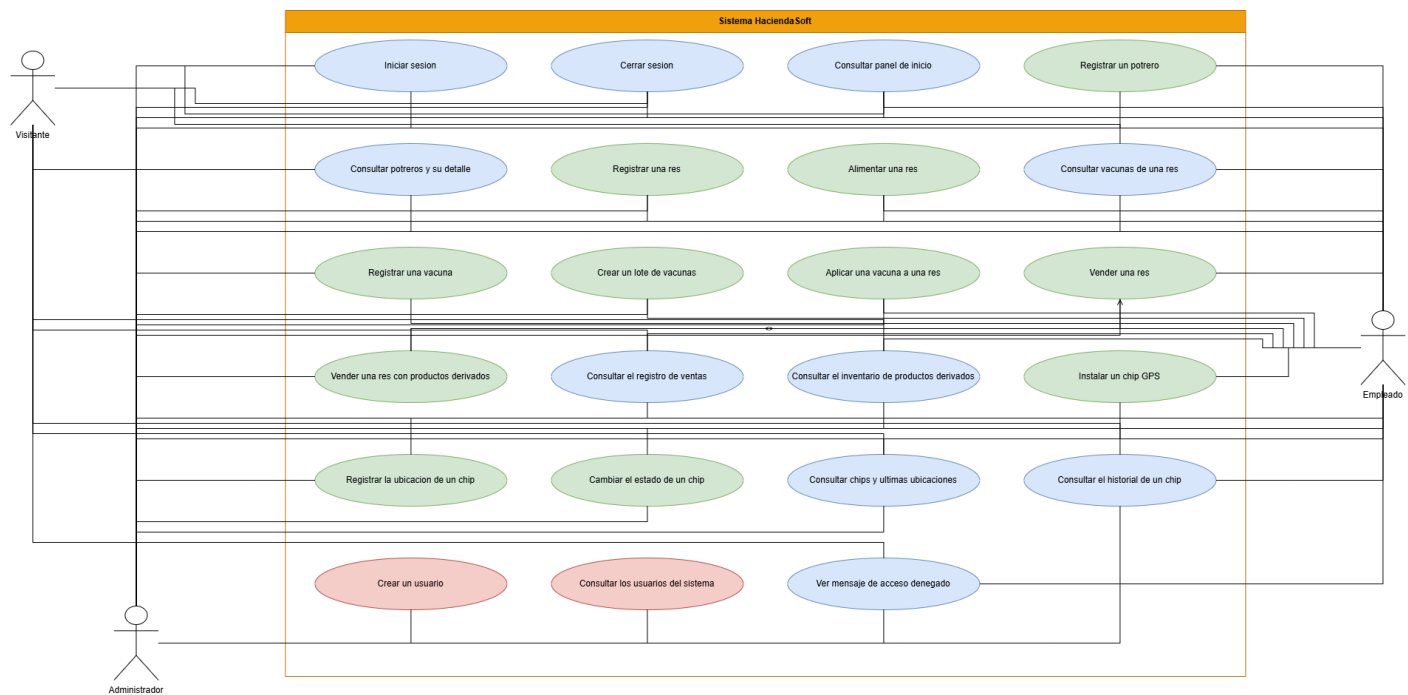
5. Análisis de riesgos

ID	Riesgo (si ocurre X, entonces Y)	Prob	Imp	Exp	Qué hacen para evitarlo	Cómo se enteran de que está pasando (el riesgo se está materializando)
R-01	Si el trabajo se retrasa, entonces la implementación puede quedar incompleta aunque el diseño esté documentado.	4	4	16	Dividir el cambio en fases y verificar cada fase antes de continuar.	Una fase termina sin código ejecutable o sin sus pruebas asociadas.
R-02	Si durante la modificación se elimina accidentalmente una función existente, entonces aparecerá una regresión.	3	4	12	Ejecutar los casos del Reto 1 después de cada cambio.	Un caso que funcionaba antes deja de funcionar.
R-03	Si una regla o mensaje cambia durante la migración, entonces cambiará el comportamiento observable.	2	5	10	Comparar las salidas antes y después.	Aparece una diferencia en las salidas para la misma entrada.
R-04	Si el orden de los handlers no es determinista, entonces la salida de consola puede cambiar.	2	4	8	Definir y probar explícitamente el orden de ejecución.	Dos ejecuciones iguales producen líneas en distinto orden.
R-05	Si un creador concreto no cumple correctamente el proceso común, entonces el nuevo subtipo puede comportarse de forma diferente.	2	4	8	Probar cada nuevo creador con los casos completos.	Un subtipo no puede completar una operación que otros subtipos sí realizan.
R-06	Si el modelo de persistencia de ventas multi-ítem no responde a las consultas necesarias, entonces será necesario modificar nuevamente el repositorio.	3	3	9	Validar primero las consultas que necesita SC-1.	Una operación requiere múltiples lecturas adicionales o no puede reconstruir la venta correctamente.
R-07	Si los nuevos registros se agregan antes de la inicialización existente, entonces puede cambiar el comportamiento del arranque.	2	4	8	Añadir los nuevos registros al final de <code>Program.cs</code> .	Error de arranque, duplicación o cambio en los datos iniciales.
R-08	Si se incorporan demasiadas abstracciones, entonces el diseño puede resultar más difícil de mantener que el problema original.	2	3	6	Mantener solamente cuatro patrones adoptados y exigir que cada uno tenga un punto de dolor asociado.	El equipo no puede explicar qué problema concreto resuelve una nueva clase o abstracción.

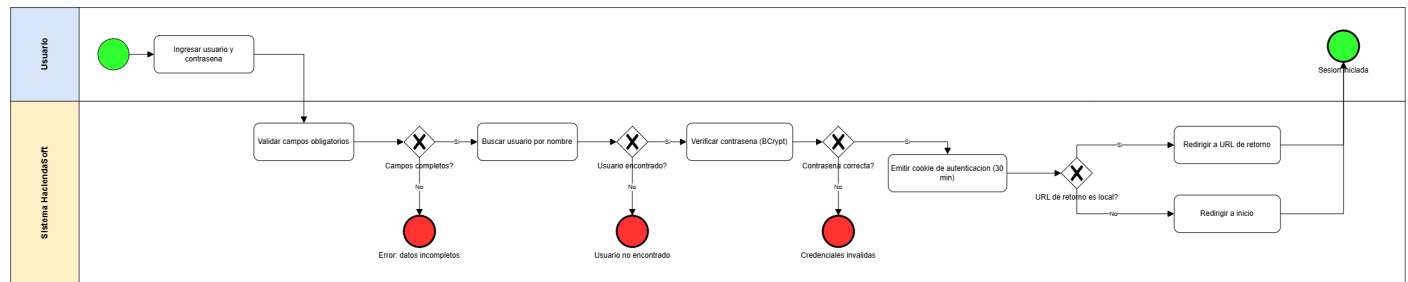
6. Vistas de la arquitectura

6.1. Vista para el negocio

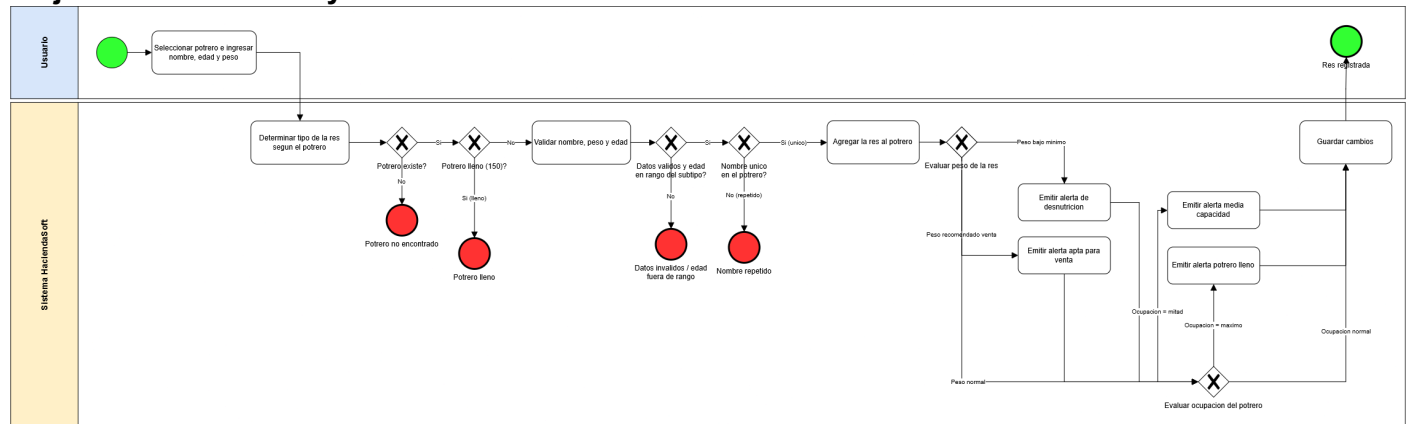
Casos de uso



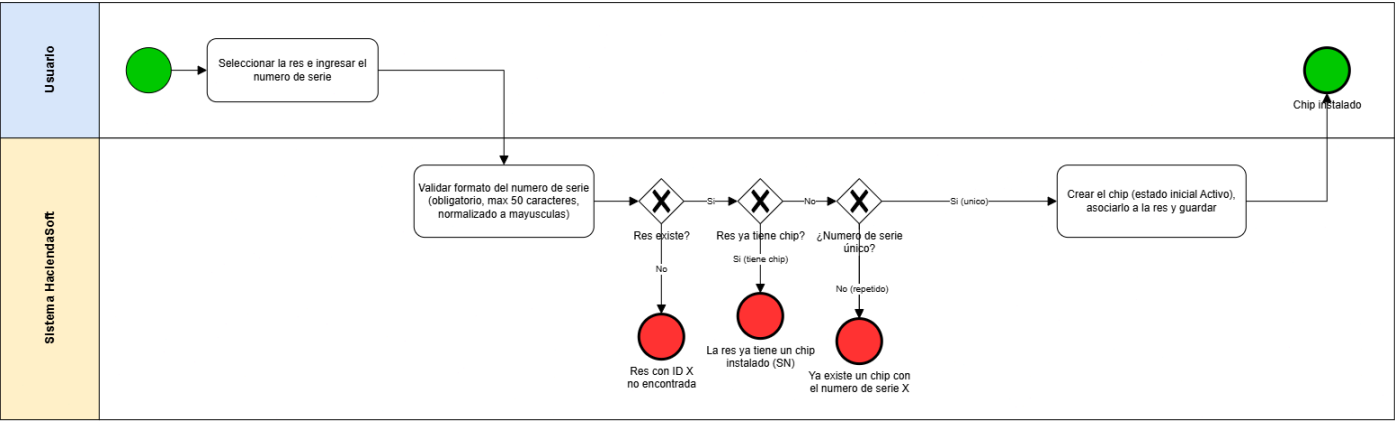
Flujo Inicio de Sesión



Flujo creación Potrero y res



Flujo Chip



¿Qué problema estamos resolviendo?

Hoy, algunos cambios aparentemente sencillos requieren modificar varias partes del sistema.

Por ejemplo, incorporar una nueva variedad de animal puede requerir abrir alrededor de seis archivos. Esto aumenta el tiempo necesario para realizar el cambio y la posibilidad de que una parte quede sin actualizar.

Además, el sistema actualmente está preparado para vender una res, pero la nueva necesidad es poder vender también los productos que se obtienen de ella.

¿Qué vamos a hacer?

Vamos a reorganizar internamente el sistema para que:

- agregar una nueva variedad de animal requiera menos modificaciones;
- una venta pueda incluir varios productos;
- nuevas reacciones puedan incorporarse sin modificar las operaciones existentes;
- las reglas repetidas estén concentradas.

¿Qué no cambia?

No se cambian:

- las pantallas;
- los mensajes actuales;
- los cálculos;
- la forma en que el usuario utiliza las operaciones existentes;
- la tecnología utilizada.

¿Qué gana el negocio?

Situación actual	Resultado esperado
Agregar una variedad nueva requiere varios cambios.	El cambio se concentra en pocos puntos.
Una venta está pensada para una sola res.	Una venta puede contener la res y sus derivados.
Una reacción nueva puede exigir modificar operaciones existentes.	Se puede agregar una nueva reacción sin alterar esas operaciones.
Una regla puede estar repetida.	Se busca una única definición para las reglas comunes.

¿Qué cuesta?

El cambio requiere trabajo de desarrollo y agrega algunos componentes internos.

No requiere:

- cambiar de lenguaje;
- contratar otra infraestructura;
- cambiar las pantallas;

- cambiar la base tecnológica.
-

6.2. Vista para el equipo de desarrollo

[Se encuentran en /diagramas con sus correspondientes evoluciones]

Deuda técnica pendiente

Quedan explícitamente pendientes:

- **P-05:** identidad durante la rehidratación, si se decide separarlo del alcance actual.
 - **P-07:** RBAC.
 - **P-08:** sensibilidad del arranque.
 - **P-09:** duplicación de `Serializar()`.
 - **P-14:** moneda COP.
 - La decisión definitiva de persistencia para las ventas multi-ítem.
 - Composite, mientras las ventas sigan siendo listas planas.
-

7. Conclusiones

El objetivo de esta evolución no es agregar patrones por sí mismos. El objetivo es reducir el costo de los cambios que actualmente requieren modificar demasiados lugares.

Los cuatro patrones adoptados se relacionan directamente con los problemas encontrados:

- **Factory Method** → creación de nuevos tipos.
- **Template Method** → procesos repetidos.
- **Builder** → ventas con varios elementos.
- **Observer** → nuevas reacciones ante eventos.

El diseño mantiene el comportamiento existente y deja explícitamente identificadas las partes que no se intervienen en este ciclo.

La evolución propuesta busca que el sistema pueda responder mejor a las solicitudes futuras sin aumentar innecesariamente la complejidad. Cada patrón tiene un problema concreto asociado, una alternativa descartada y un costo explícito, de manera que la decisión arquitectónica pueda ser revisada posteriormente por otro desarrollador.